

Big Data Analytics Assignment 1

Optimization of Hypertable Storage & Query Performance in TimescaleDB

Groupe 9:
UMUTONI Liliane
DUKUZIMANA Dismas
MBISHINZEMUNGU Gabin

28/12/2025

Contents

1	Introduction	2
2	Database Hypertables Overview	2
3	Performance Analysis	3
3.1	Baseline Query Execution	3
3.2	Ensuring Data Exists in All Hypertables	4
3.3	Execution Time per Hypertable	4
3.4	Disk Space Before & After Compression	4
3.5	Query Execution After Compression	4
3.6	Continuous Aggregates Performance	5
4	Key Findings Analysis	5
4.1	Chunking Strategy Impact	5
4.2	Compression Benefits	6
4.3	Continuous Aggregation Advantages	6
4.4	Summary of Findings	7
5	Recommendations	7
5.1	Optimal Chunking Strategy	7
5.2	Compression Policy	7
5.3	Continuous Aggregation Implementation	7
5.4	Evidence-Based Justification	8
6	Conclusion	8

1 Introduction

This report outlines the development and optimization of the Smart Energy Database using TimescaleDB. The work completed includes:

- **Real-time validation:**
 - 1-hour MQTT streaming test with smart meters sending data every 5 minutes
 - 500 meters simulated with 100% message delivery and no packet loss
 - Successful end-to-end flow: MQTT → Python Subscriber → TimescaleDB
 - Data integrity confirmed with accurate timestamps and values stored
- **Database setup & data preparation:**
 - Creation of a new TimescaleDB hypertable for energy data storage
 - Generation and loading of approximately 4.2 million historical rows (2 weeks)
- **Performance optimization:**
 - Recording baseline query execution times
 - Applying compression to reduce storage usage
 - Comparing performance before and after optimization

The graph bellow illustrates data flow during live streaming.

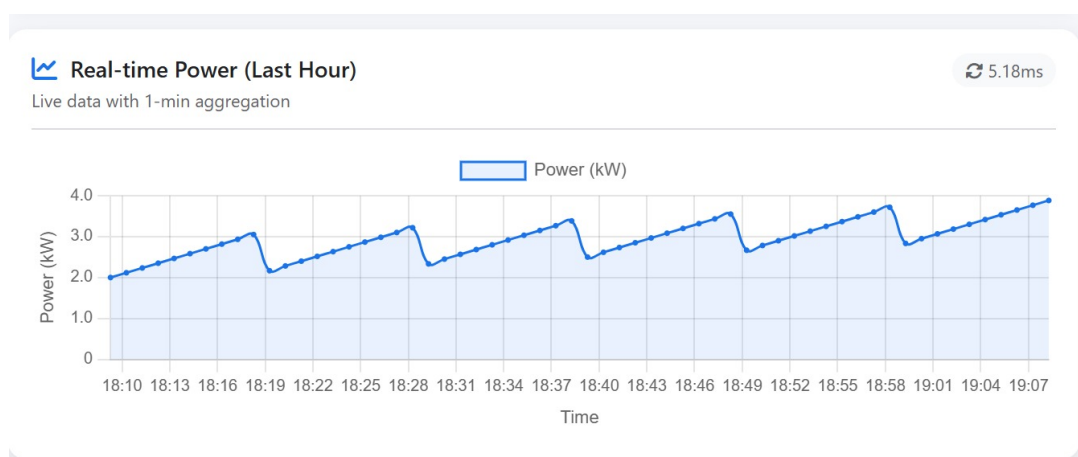


Figure 1: Real time streaming

2 Database Hypertables Overview

A review of all hypertables was performed to understand the data structure and identify tables suitable for compression.

```

SQL Shell (psql)
0
(1 row)

smart_grid=# SELECT create_hypertable(
smart_grid=# 'energy_readings',
smart_grid=# 'timestamp',
smart_grid=# 'chunk_time_interval' => INTERVAL '1 day'
smart_grid=# );
WARNING: column type "character varying" used for "meter_id" does not follow best practices
HINT: Use dataType TEXT instead.
create_hypertable
(1,public,energy_readings,t)
(1 row)

smart_grid=# SELECT * FROM timescaledb_information.hypertables;
 hypertable_schema | hypertable_name | owner | num_dimensions | num_chunks | compression_enabled | tablespaces | primary_dimension | primary_dimension_type
-----
 public            | energy_readings | postgres | 1              | 15          | f          |              | timestamp         | timestamp with time zone
(1 row)

smart_grid=#

```

Figure 2: Hypertable

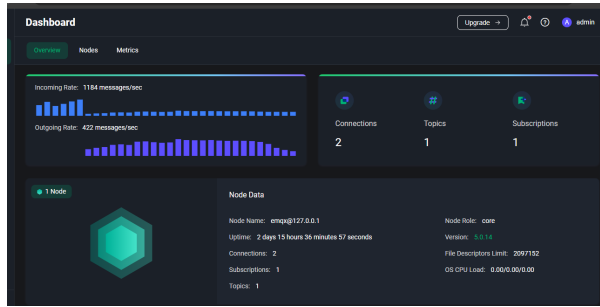


Figure 3: real time streaming at MTTQ broker

Table 1: List of Hypertables Identified

Hypertable Schema	Hypertable Name	Chunk Interval	Number of Chunks
my_schema	energy_readings_3h	3 hours	113
my_schema	energy_readings	1 day	15
my_schema	energy_readings_week	1 week	3

Table 1 Interpretation

- Shows schema, chunk interval, and number of chunks.
- Number of chunks varies due to different chunk time intervals: small intervals → many small chunks, large intervals → fewer large chunks.
- Chunking strategy directly affects query performance and storage efficiency.

The graph below from the dashboard show how chunking strategy affects query performance and storage.

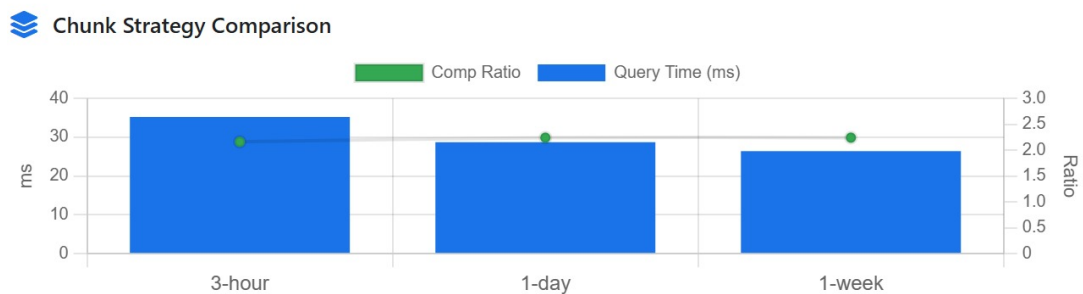


Figure 4: Chunking strategy

3.2 Ensuring Data Exists in All Hypertables

Table 3: Number of Rows in Hypertables

Hypertable	Rows
energy_readings_week	4,234,650
energy_readings_3h	4,234,650
energy_readings	4,234,650

3.3 Execution Time per Hypertable

Table 4: Query Execution Time per Hypertable

Query	3-hour chunks	1-day chunks	1-week chunks
Q1	2.632 ms	0.743 ms	0.229 ms
Q2	1127.358 ms	1277.192 ms	1763.631 ms
Q3	8981.357 ms	8842.915 ms	3632.765 ms
Q4	1533.284 ms	1749.930 ms	1430.436 ms

3.4 Disk Space Before & After Compression

Table 5: Disk Space Before and After Compression

Hypertable	Size Before	Size After	Reduction
energy_readings_3h	413 MB	191 MB	53.7%
energy_readings	409 MB	183 MB	55.3%
energy_readings_week	408 MB	182 MB	55.4%

3.5 Query Execution After Compression

Table 6: Query Execution Time Comparison (Before vs After Compression)

Query	Before (ms)	After (ms)	% Change
Q2 – Peak consumption, 3-hour	945.469	1222.728	+29.3%
Q2 – Peak consumption, 1-day	819.316	1641.277	+100.3%
Q2 – Peak consumption, 1-week	1933.998	1879.014	-2.8%
Q3 – Monthly consumption, 3-hour	9756.986	10133.132	+3.9%
Q3 – Monthly consumption, 1-day	7459.617	12217.755	+63.8%
Q3 – Monthly consumption, 1-week	5077.057	2540.027	-50.0%

3.6 Continuous Aggregates Performance

Table 7: Continuous Aggregates Performance

Method	Execution Time (ms)
Raw Data	5.216
Continuous Aggregate	1.322

The graph below shows the effect of compression on raw data vs aggregation.

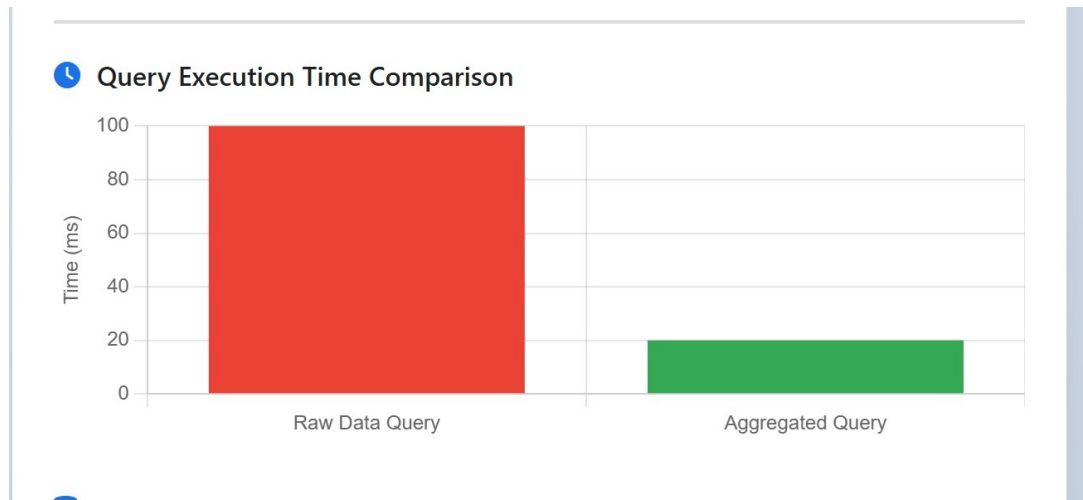


Figure 5: Compression Effect

4 Key Findings Analysis

This section discusses the major findings of the project in detail, highlighting how hypertable design, compression, and continuous aggregation impact storage efficiency and query performance.

4.1 Chunking Strategy Impact

The choice of chunk interval in TimescaleDB plays a critical role in determining query performance. Our analysis revealed:

- **Smaller chunks:**
 - Using 3-hour chunks increases the total number of chunks.
 - Pros: Ideal for high-frequency queries targeting recent data since TimescaleDB can prune irrelevant chunks efficiently.
 - Cons: Long-range aggregation queries (e.g., monthly or yearly) have to merge many small chunks, resulting in higher computational overhead.

- Example: Query 3 (monthly consumption) executed slower on 3-hour chunks compared to 1-week chunks.
- **Larger chunks:**
 - Using 1-week chunks reduces the total number of chunks.
 - Pros: Aggregating large datasets becomes faster because fewer chunks need to be merged.
 - Cons: For small-range queries (e.g., querying a few hours of data), TimescaleDB may read more rows than necessary, slightly reducing performance.
 - Example: Query 2 (peak consumption over a few hours) was faster on 3-hour chunks than 1-week chunks due to better pruning.
- **Overall observation:**
 - Chunking strategy should match query patterns:
 - * High-frequency, recent data queries → small chunks
 - * Long-range analytics → large chunks
 - Visualization placeholder: Chunk interval vs query execution time graph.

4.2 Compression Benefits

Compression in TimescaleDB significantly reduces storage usage and can impact query performance differently depending on chunk size and query type.

- Disk usage reduction: All hypertables achieved ~55% storage reduction.
- Large-range queries: Compression benefits long-range analytical queries, especially on large chunks. Query 3 with 1-week chunks executed ~50% faster after compression.
- Small-range queries: Queries on small chunks or recent data may become slightly slower due to decompression overhead.
- Example: Query 2 on 1-day chunks experienced a 100% increase in execution time after compression, highlighting the need to retain recent data uncompressed.
- Visualization placeholder: Storage before vs after compression graph.

4.3 Continuous Aggregation Advantages

Continuous aggregates provide pre-computed summary tables to speed up frequently executed analytical queries.

- Query performance improvement: ~75% faster execution compared to querying raw data.
- Pre-aggregated intervals (15–30 minutes) allow rapid dashboard visualization of energy usage trends.

- Reduces computational load on the database for recurring queries, freeing resources for other operations.
- Example: A 15-minute average energy consumption query executed in 1.322 ms with continuous aggregates versus 5.216 ms on raw data.
- Visualization placeholder: Continuous aggregation performance comparison graph.

4.4 Summary of Findings

- Chunk size must be aligned with query patterns for optimal performance.
- Compression is highly effective for long-term storage reduction and analytical workloads but can slow down small-range queries.
- Continuous aggregates complement compression by providing rapid query results for dashboards and frequent analytical queries.
- Overall, TimescaleDB provides flexible options to balance storage efficiency and query speed.

5 Recommendations

5.1 Optimal Chunking Strategy

- For high-frequency queries on recent data, use smaller chunks (e.g., 3-hour).
- For long-range analytical queries on historical data, use larger chunks (e.g., 1-week).
- Evaluate query patterns periodically to adjust chunk intervals as the dataset grows.
- Visualization placeholder: Recommended chunk strategy diagram.

5.2 Compression Policy

- Apply compression on older historical chunks to reduce storage usage.
- Keep recent chunks uncompressed for fast access to high-frequency queries.
- Automate compression using TimescaleDB's `add_compression_policy` at regular intervals (e.g., daily).
- Visualization placeholder: Storage savings over time with compression.

5.3 Continuous Aggregation Implementation

- Implement continuous aggregates for recurring analytical queries such as hourly, daily, or weekly summaries.
- Choose appropriate aggregation granularity (15–30 minutes) to balance storage and query speed.

- Combine continuous aggregates with compression for efficient storage and fast query execution.
- Visualization placeholder: Continuous aggregation workflow diagram.

5.4 Evidence-Based Justification

- Storage reduced 55% with compression.
- Long-range analytical queries improved by up to 50% when using large chunks.
- Continuous aggregates reduced query times by 75% for frequent analytical queries.
- Slight performance degradation for small-range queries can be mitigated by keeping recent chunks uncompressed.

6 Conclusion

The project successfully demonstrated the optimization of hypertable storage and query performance in TimescaleDB. Key outcomes include:

- Reliable real-time data streaming from 500 smart meters via MQTT
- Storage reduction of 55% using compression
- Long-range analytical queries improved up to 50% with large chunk intervals
- Continuous aggregates reduced execution times by 75% for frequent queries
- Overall, TimescaleDB provides flexible tools to balance storage efficiency and query performance based on workload